

Team 3 — RAG Legal AI (Python) Production Roadmap

Persistent Vector Store · GDPR Corpus · Conversation Memory · Anti-hallucination

RAG Team

YourCrawl Dark Pattern Auditor

Generated: 16 August 2026

RAG Legal AI Service (Python/FastAPI) Production Roadmap

Assigned To: RAG Team Lead

Priority: HIGH — Legal compliance intelligence is a key differentiator

What Exists Today (Current State Assessment)

Component	File	Status
FastAPI Server	api.py	Functional on port 8002
ChromaDB Vector Store	chroma_db/	Local filesystem, not scalable
RAG Agent	rag_agent.py	Functional Q&A + compliance check
Ingestion Pipeline	ingest.py	Loads 3 legal documents
Embedding Model	all-MiniLM-L6-v2	Sentence-transformers
Legal Documents	data/	DPDP Act, EU AI Act, Consumer Protection Act
Audit Q&A	/query-with-audit	Working
Conversation Memory	None	MISSING
Citation Verification	None	MISSING
Async Streaming	None	MISSING
CORS Security	Wildcard *	SECURITY RISK
Gemini Key Rotation	Across 3 keys	Functional
Confidence Scoring	None	MISSING
Legal Update Mechanism	None	MISSING

1. Local ChromaDB Cannot Scale

File: `ingest.py`, `rag_agent.py` ChromaDB is stored in `./chroma_db/` on the local filesystem. This means:

- If the service restarts and the directory is lost, all embeddings are gone
- Cannot run multiple instances of the RAG service in parallel
- Not compatible with containerized/cloud deployments where the filesystem is ephemeral
- Data loss on any crash or redeploy

Fix: Migrate to a hosted vector database (Pinecone, Weaviate, or Chroma Cloud) or persist ChromaDB to a mounted volume backed by cloud storage.

2. No Conversation Memory (Chat Context Lost)

The `/query-with-audit` endpoint accepts a `session_id` field but the `rag_agent.py` implementation completely ignores it. Every question is answered in isolation — if a user asks a follow-up question like "Can you explain that more?", the agent has no idea what "that" refers to. For a product positioning itself as an AI assistant, this is a dealbreaker for user experience.

3. Only 3 Legal Documents Indexed

File: `ingest.py` The legal corpus contains only:

- `dpdp_act.txt` (India DPDP Act 2023)
- `eu_ai_act.txt` (EU AI Act 2024)
- `consumer_protection.txt` (Consumer Protection Act 2019 + CCPA Guidelines 2023)

Missing critical regulations:

- GDPR (General Data Protection Regulation) — most referenced globally
- FTC Act Section 5 (USA) — referenced in ML compliance mapper
- EU Digital Services Act (DSA) — referenced in ML compliance mapper
- COPPA (Children's Online Privacy Protection Act)
- UK GDPR
- Brazil LGPD

The ML service references GDPR and DSA in its compliance annotations, but the RAG service cannot answer questions about them because they are not indexed. This is a direct inconsistency between services.

4. No Citation Confidence Scoring

The RAG agent retrieves chunks from ChromaDB and returns them as sources, but there is no quality check on how relevant those chunks are. Low-relevance chunks pollute the response with incorrect or tangential legal citations, which could mislead users into thinking something is a legal violation when it is not.

5. Wildcard CORS in Production

File: `api.py` line 68

```
allow_origins=["*"],
```

The RAG service is a sensitive legal reasoning engine. It should only be callable by the Backend Node.js service, not from browsers directly.

6. No Async / Streaming Support for Long LLM Responses

Gemini generation for complex compliance checks can take 10-20 seconds. The current implementation blocks the HTTP connection for the full duration. Users see a blank screen with no feedback.

System Architecture: Target State

```
Backend Node.js
|
v
[RAG FastAPI - port 8002]
|
+-- /query                -> Legal Q&A (free-form)
+-- /query-with-audit     -> Audit-grounded Q&A (with session memory)
+-- /compliance-check     -> Pattern-to-law mapping
+-- /stream/query-with-audit -> SSE streaming response
|
v
[Retrieval Layer]
|
+-- [ChromaDB / Pinecone] <- Persistent vector store
|      ^
|      | ingest
+-- [Legal Document Corpus] (10+ laws)
|
v
[Gemini LLM Generation] <- Key rotation across 3+ keys
|
v
Structured JSON Response with citations + confidence scores

[Redis] <- Session memory (conversation history by session_id)
```

Feature Tasks to Build

Feature 1 — Persistent Vector Store (HIGHEST PRIORITY)

Why: Local ChromaDB is not production-safe. Any restart loses all embeddings and requires re-ingestion.

Option A (Minimal Change): Mount ChromaDB to a Persistent Volume If deploying with Docker:

```
volumes:
  - ./chroma_db:/app/chroma_db
```

Ensure ChromaDB uses an absolute path from an environment variable:

```
CHROMA_PATH = os.getenv('CHROMA_PATH', './chroma_db')

client = chromadb.PersistentClient(path=CHROMA_PATH)
```

Option B (Production Grade): Migrate to Chroma Cloud or Pinecone

```
import chromadb

client = chromadb.HttpClient(
    host=os.getenv('CHROMA_HOST'),
    port=int(os.getenv('CHROMA_PORT', 8000)),
    ssl=True
)
```

Acceptance Criteria: ChromaDB data survives service restarts. Re-ingestion is not required on every deployment.

Feature 2 — Conversation Memory with Session ID

Why: `session_id` is accepted in the API contract but completely ignored. Users cannot have multi-turn conversations. **Implementation Steps:**

14. Install: `pip install redis`

15. Create `memory/conversation_store.py`:

```
import redis, json

r = redis.Redis(host=os.getenv('REDIS_HOST', 'localhost'), decode_responses=True)

class ConversationStore:

    MAX_HISTORY = 10 # keep last 10 turns

    TTL = 3600 # 1 hour

    def get_history(self, session_id: str) -> list[dict]:

        val = r.get(f"session:{session_id}")

        return json.loads(val) if val else []

    def append(self, session_id: str, role: str, content: str):

        history = self.get_history(session_id)

        history.append({"role": role, "content": content})

        history = history[-self.MAX_HISTORY:] # keep only last N turns

        r.setex(f"session:{session_id}", self.TTL, json.dumps(history))
```

16. In `rag_agent.py` `query_with_audit()` method:

- Load conversation history by `session_id` - Prepend history to the Gemini prompt - Append the new Q&A pair to history after generation

17. Add `POST /api/sessions/{session_id}/clear` endpoint to reset conversation.

Acceptance Criteria: A user can ask a follow-up question that references their previous question and receive a contextually correct answer.

Feature 3 — Expand Legal Corpus (Add GDPR and DSA)

Why: The ML service references GDPR and EU DSA in compliance annotations, but the RAG agent cannot answer questions about them. This is a critical inconsistency that will confuse users. **Implementation Steps:**

18. Add these legal documents to `rag/data/`:

- `gdpr.txt` — Full GDPR text (official EUR-Lex source) - `eu_dsa.txt` — EU Digital Services Act - `ftc_act_section5.txt` — FTC Act Section 5 guidelines on deceptive practices - `coppa.txt` — COPPA (if targeting products used by children)

19. Update `ingest.py` to discover and ingest all `.txt` files in `data/`:

```
import glob

doc_files = glob.glob('data/*.txt')

for doc_path in doc_files:

    ingest_document(doc_path)
```

20. Add metadata tagging so each chunk knows its source regulation and jurisdiction:

```

metadata = {
    'source': 'GDPR',
    'jurisdiction': 'EU',
    'article': 'Article 7',
    'year': 2018
}

```

21. Re-run ingestion: `python ingest.py`

Acceptance Criteria: Questions about GDPR, EU DSA, and FTC Act receive accurate, cited answers. The health endpoint lists all indexed regulations.

Feature 4 — Citation Confidence Scoring and Relevance Filtering

Why: Low-relevance retrieved chunks produce hallucinated or misleading legal citations, which is dangerous for a compliance product. **Implementation Steps:**

22. Add relevance threshold filtering in `rag_agent.py`:

```

def retrieve(self, query: str, min_relevance: float = 0.4) -> list[dict]:
    results = self.collection.query(
        query_texts=[query],
        n_results=10,
        include=['documents', 'metadatas', 'distances']
    )

    chunks = []

    for doc, meta, dist in zip(
        results['documents'][0],
        results['metadatas'][0],
        results['distances'][0]
    ):
        relevance = 1 - dist # ChromaDB uses cosine distance

        if relevance >= min_relevance:
            chunks.append({
                'text': doc,
                'metadata': meta,
                'relevance': round(relevance, 3)
            })

    return sorted(chunks, key=lambda x: x['relevance'], reverse=True)[:5]

```

23. Include `relevance` scores in the API response `SourceChunk` model.

24. If no chunks pass the relevance threshold, return: "I could not find sufficient legal context to answer this question confidently."

Acceptance Criteria: API responses include relevance scores for each citation. No chunks below 0.4 relevance are included in responses.

Feature 5 — Streaming Responses via SSE

Why: Gemini generation for complex compliance checks takes 10-20 seconds. Users need feedback.

Implementation Steps:

25. Add FastAPI streaming endpoint:

```
from fastapi.responses import StreamingResponse
import asyncio

@app.post("/stream/query-with-audit", tags=["Streaming"])
async def stream_query_with_audit(req: AuditQueryRequest):
    async def generate():
        async for chunk in _agent.stream_query_with_audit(req.question, req.audit_data):
            yield f"data: {json.dumps({'chunk': chunk})}\n\n"
        yield "data: [DONE]\n\n"

    return StreamingResponse(generate(), media_type="text/event-stream")
```

26. Implement `stream_query_with_audit()` in `rag_agent.py` using Gemini's streaming API.

27. Add a non-streaming fallback for clients that do not support SSE.

Acceptance Criteria: The streaming endpoint delivers the first token within 2 seconds. The full response streams progressively.

Feature 6 — Hallucination Prevention and Response Grounding

Why: LLMs can fabricate legal citations. For a compliance product, this is unacceptable and creates legal liability. **Implementation Steps:**

28. Add explicit grounding instruction to all system prompts:

```
You MUST only cite legal provisions that appear verbatim in the provided context.
If you cannot find relevant legal context, say: "I do not have sufficient legal
information to answer this question. Please consult a qualified legal professional."
Never invent article numbers, section references, or clause text.
```

29. After generation, verify that cited article numbers exist in the retrieved chunks:

```
def verify_citations(answer: str, retrieved_chunks: list[dict]) -> bool:
    chunk_text = ' '.join(c['text'] for c in retrieved_chunks)

    # Extract article references from answer
    # Check each reference appears in chunk_text

    return all_citations_grounded
```

30. If verification fails, fall back to a safer response: "Based on the indexed legal documents..."

Acceptance Criteria: Every article/section cited in an answer is present in the retrieved chunks. Ungrounded responses are flagged or suppressed.

Feature 7 — Legal Document Update Pipeline

Why: Laws change. The EU AI Act enforcement timeline changes. DPDP Act gets amendments. The RAG service needs a way to update its legal corpus without manual re-ingestion. **Implementation Steps:**

31. Add `POST /api/admin/ingest` endpoint (admin-only, secured with separate key):

```
@app.post("/api/admin/ingest", dependencies=[Depends(verify_admin_key)])
async def ingest_document(file: UploadFile):
    content = await file.read()

    # Save to data/ directory and trigger re-ingestion
    save_and_ingest(filename=file.filename, content=content)

    return {"status": "ingested", "chunks_added": chunk_count}
```

32. Add `GET /api/admin/corpus` endpoint that lists all indexed documents with their chunk counts and last updated timestamps.

33. Add `DELETE /api/admin/corpus/{document_name}` to remove outdated documents.

Acceptance Criteria: An admin can add a new legal PDF/text file via API without restarting the service.

Feature 8 — CORS Hardening and Production Security

Implementation Steps:

34. Replace wildcard CORS:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=[os.getenv('ALLOWED_BACKEND_ORIGIN', 'http://localhost:3000')],
    allow_methods=["POST", "GET"],
    allow_headers=["X-API-Key", "Content-Type"],
)
```

35. Add startup check for required secrets:

```
required_env = ['RAG_API_KEY', 'GEMINI_API_KEY']
missing = [k for k in required_env if not os.getenv(k)]
if missing:
    raise RuntimeError(f"Missing required environment variables: {missing}")
```

36. Add request body size limit middleware.

37. Ensure `RAG_API_KEY` is at least 32 characters long (current key is 64 hex chars — good).

Acceptance Criteria: RAG service rejects requests from unauthorized origins. Service fails to start if secrets are missing.

Legal Accuracy Requirements

For a compliance product, the RAG service must meet minimum legal accuracy benchmarks before launch:

Metric	Target
Correct law identified	>= 90% on benchmark Q&A dataset
Correct article cited	>= 80% on benchmark Q&A dataset
Hallucinated citations	<= 5%
Relevance of retrieved chunks	>= 0.5 average cosine similarity

Create a benchmark Q&A dataset with 50 manually verified question-answer pairs to measure these metrics.

Delivery Timeline

Week	Milestone
Week 1	Feature 1 (Persistent Vector Store) + Feature 3 (Expand Legal Corpus)
Week 2	Feature 2 (Conversation Memory) + Feature 4 (Citation Confidence)
Week 3	Feature 5 (Streaming) + Feature 8 (Security Hardening)
Week 4	Feature 6 (Hallucination Prevention) + Feature 7 (Update Pipeline)

New Dependencies to Add

```
redis>=5.0

pinecone-client>=3.0 (if migrating from local Chroma)

pytest>=8.0

pytest-asyncio>=0.23

httpx>=0.27  (for async testing)
```

Definition of "Production Ready" for RAG Service

- ☐ ChromaDB persisted to reliable storage (not local ephemeral disk)
- ☐ GDPR and EU DSA documents indexed
- ☐ Conversation memory working with session IDs
- ☐ Citation confidence scores included in every response

- ☐ Relevance filtering active (no sub-0.4 chunks in responses)
- ☐ Streaming endpoint working
- ☐ Hallucination prevention prompt active
- ☐ CORS restricted to Backend origin only
- ☐ All secrets validated at startup
- ☐ Legal accuracy benchmark: $\geq 90\%$ correct law identification
- ☐ Admin ingestion endpoint for legal document updates